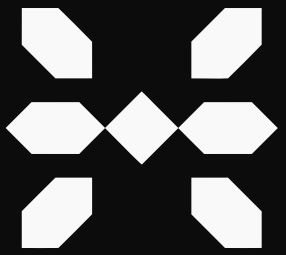
HOOX · AXIS · DOCUMENTATION



EDGE OPERATORS · WORKER AUTHORS

WORKER MANUAL

Cloudflare Worker, Durable Objects, KV, D1, R2, auth

Open charting PWA — own the axes, swap the engine

CONTENTS

01 Overview

02 Runtime

03 Durable Objects

04 Data Plane

05 Auth

06 Bindings

07 Mcp

OVERVIEW

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Worker" description: "Cloudflare® Worker for AXIS: /api/run proxy, keys, scripts, usage, and Durable Object stream relay."

WORKER

Abstract

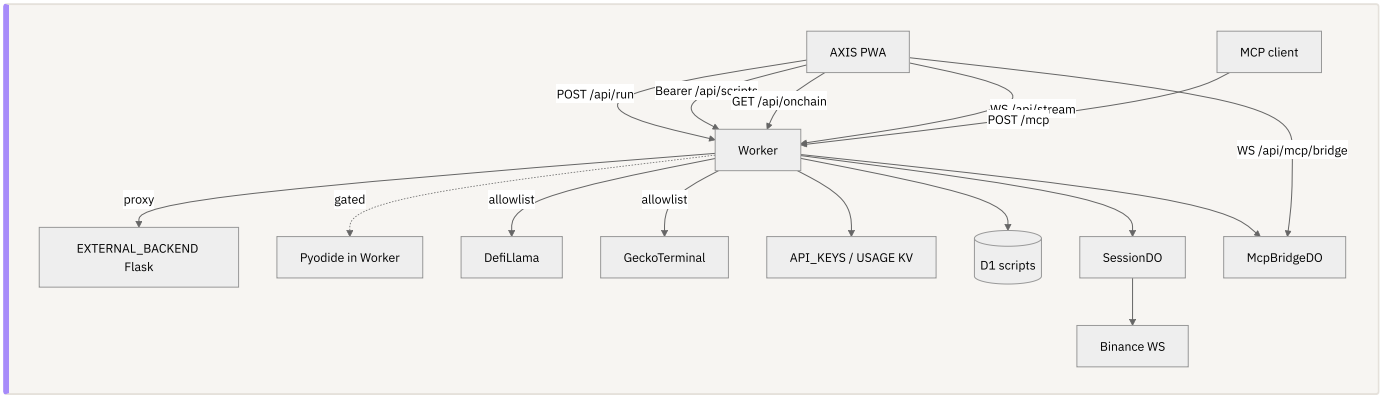
The AXIS Cloudflare® Worker (`worker/`) is the **edge data plane** for the PWA: JSON APIs, optional script library (D1), API keys (KV), usage meters, on-chain proxy, and a WebSocket session relay (Durable Objects). It is **not** a full Pine Script™ interpreter in production today.

Honest runtime fact: `POST /api/run` **proxies to** `EXTERNAL_BACKEND` (typically Flask) unless `PYODIDE_IN_WORKER=enabled` and the in-worker path succeeds. That path is implemented and **off by default** (`disabled` in `worker/wrangler.toml.example`). It is not a planned feature. See Runtime and `worker/RUNTIME.md` .

Security (2.0.1+): gated `/api/run` (auth when `API_KEYS` / `REQUIRE_RUN_AUTH`), rate limits + body caps, fail-closed Worker auth when D1 is bound without `API_KEYS` KV, product-scoped CORS (not open `*.pages.dev`), OAuth env client ids over body `clientId`. See Auth and CORS.

Operator path: AXIS CLI — `axis setup` · `axis deploy worker` · `axis health`.

Conceptual model



Infrastructure names (frozen)

Surface	Value
Wrangler Worker script	**`worker-axis`** — canonical `https://worker.axis.hoox.sh`
Pages project	**`axis`** — canonical `https://axis.hoox.sh`
npm package	`axis-worker`
Health JSON `service`	`worker-axis`
Product brand	**AXIS**

Pages `--project-name` is `axis`; custom domain is `axis.hoox.sh`. Worker URL is `https://worker.axis.hoox.sh`.

Interface surface

Path	Method	Role
`/`, `/health`	GET	Health + feature flags (`scripts`, `d1`, `keys`, `onchain`)
`/api/run`	POST	Run script (proxy / gated Pyodide)
`/api/keys`	GET/POST	Validate / create keys (`X-Admin-Token` for create)
`/api/usage`	GET	Usage stub / KV-backed counters
`/api/scripts` …	CRUD	Script library (Bearer)
`/api/onchain/…`	GET	Public on-chain proxy (DefiLlama + GeckoTerminal allowlist; see below)
`/api/stream`	WS	Session DO relay
`/mcp`	POST	MCP server (Bearer) — Worker APIs + live-app control
`/api/mcp/bridge`	WS	PWA MCP control plane

On-chain proxy routes (`worker/src/onchain.ts`)

Allowlisted **GET** only (public, no API key). Other `/api/onchain/*` paths return 404.

Worker path	Upstream	Cache TTL
`/api/onchain/health`	local (providers + cache size)	—
`/api/onchain/llama/protocols`	`https://api.llama.fi/protocols`	~10 min
`/api/onchain/llama/protocol/:slug`	`https://api.llama.fi/protocol/:slug`	~2 min
`/api/onchain/gecko/networks/:network/pools/:address/ohlcv/:timeframe`	`https://api.geckoterminal.com/api/v2/networks/.../ohlcv/...`	~60 s
`/api/onchain/gecko/search/pools`	`https://api.geckoterminal.com/api/v2/search/pools`	~120 s

Gecko validation: `network` `^[a-z0-9_]+$`; `address` EVM `0x` +40 hex **or** Solana-style base58 32–48; `timeframe` `day` | `hour` | `minute`.
OHLCV query pass-through: `aggregate`, `limit`, `currency`, `before_timestamp`. Search: `query`, `network`, `page`, `include`.

Responses may include `X-Axis-Onchain-Cache: HIT|MISS`. Product guide: On-Chain data.

Entry: `worker/src/index.ts`.

Shared HTTP plumbing (`http.ts` · `proxy-router.ts` · `rate-limit.ts`)

Per-route CORS/JSON helpers were consolidated into `worker/src/http.ts` (read vs write method sets, base vs exchange vs MCP header sets, `preflight` / `methodNotAllowed` / `proxyJson` envelopes). The on-chain and market planes share the allowlisted `ProxyRouter` (`worker/src/proxy-router.ts`): prefix claim → preflight → non-`GET` 405 → literal health path → ordered route table → family `NOT_FOUND`. Rate limiting is shared via `worker/src/rate-limit.ts`. Pinned by `worker/tests/http.test.ts`, `proxy-router.test.ts`, `rate-limit.test.ts`.

Local entrypoints

```
cd worker
npm install   # or bun install
npm run dev   # wrangler dev → http://127.0.0.1:8787
```

Makefile: `make worker-dev`, `make worker-deploy`.

Implementation status

Feature	Status
`/api/run` → `EXTERNAL_BACKEND`	**Works today**
Admin `/api/keys`	Works (KV or shape-check)
Usage KV increment on run	Works when `USAGE` bound
`SessionDO` + `/api/stream`	Implemented; binding often commented until provisioned
`POST /mcp` + `McpBridgeDO`	**Works today** when `MCP_BRIDGE` is bound
`/api/scripts` + D1 schema	Implemented; memory fallback without D1
Pyodide scaffold	`pyodide_runtime.ts` + flag; **wheel pipeline incomplete**
Response cache for `/api/run`	Deferred

Page map

Page	Topic
Runtime	`/api/run`, proxy, Pyodide plan
Durable Objects	Stream fan-out
Data plane	Scripts, D1, drafts
Auth	Bearer keys, admin token
Bindings	wrangler.toml, vars, provision
MCP server	`POST /mcp`, tools, PWA bridge

See also

- Cloudflare® deploy
- Engines
- Worker README: `worker/README.md`

RUNTIME

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Worker runtime" description: "POST /api/run — auth gate, rate limits, size caps, EXTERNAL_BACKEND proxy, feature-gated in-worker Pyodide."

WORKER RUNTIME

Abstract

`worker/src/runtime.ts` implements `handleRun` for **`POST /api/run`**. Execution preference is documented in-file:

1. **`PYODIDE_IN_WORKER=enabled`** → `tryRunInWorker` (`pyodide_runtime.ts`)
2. Else **`EXTERNAL_BACKEND`** set → HTTP proxy to `${EXTERNAL_BACKEND}/run`
3. Else **503** `NO_BACKEND` with a pointer at env vars and `worker/RUNTIME.md`

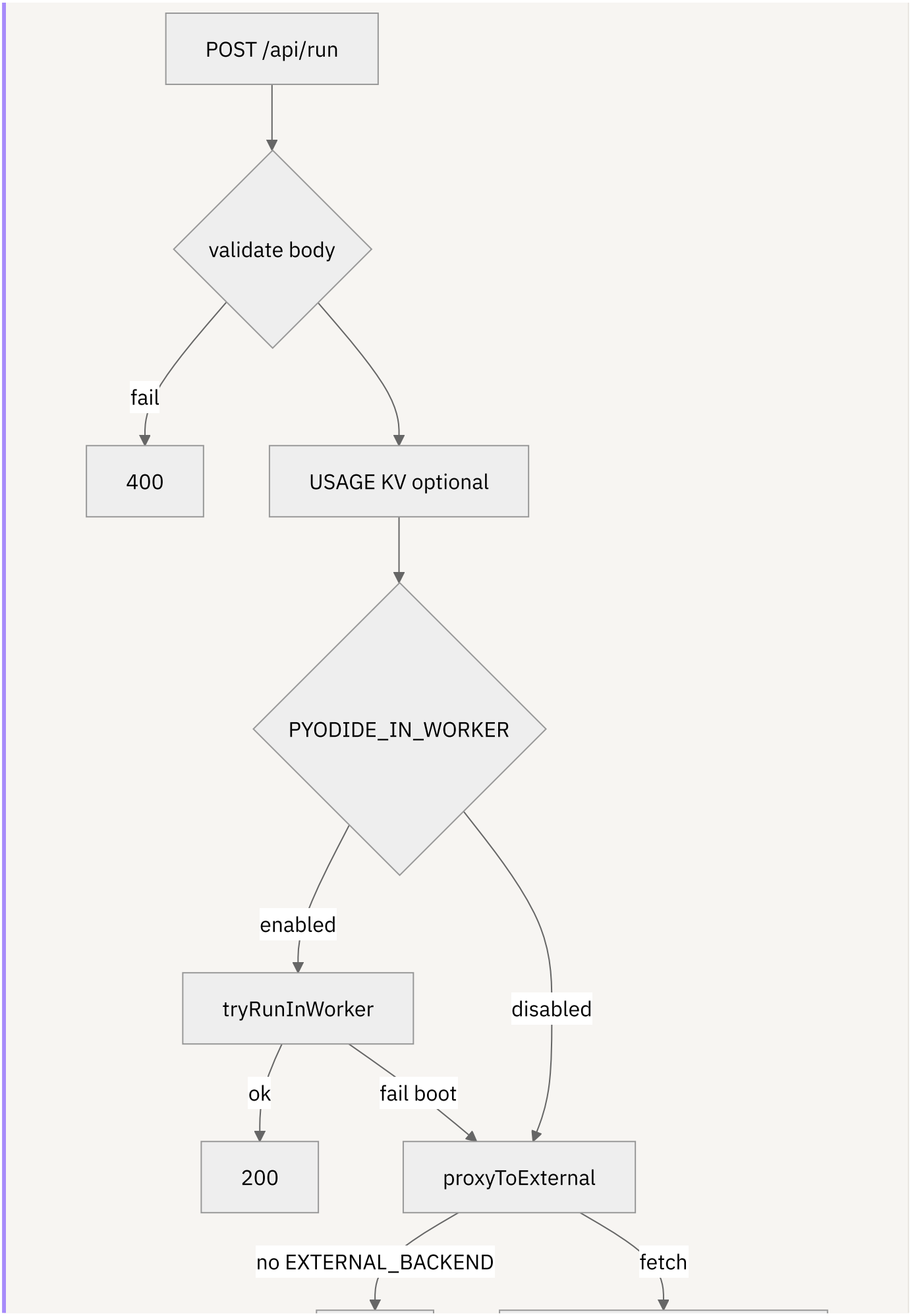
Production reality: ships as a **proxy** to Flask (or another Python host). In-worker Pyodide is a **scaffold**, not a complete wheel-upload pipeline.

Auth & abuse controls (2.0.1+)

Control	Behavior
Auth gate	When `API_KEYS` KV is bound **or** `REQUIRE_RUN_AUTH=1`, `requireApiKey` is mandatory
Rate limit	30 requests / minute per IP (and per key when authenticated)
Script cap	Max **512 KiB** source chars
Bars cap	Max **50 000** OHLCV rows per run
Upstream timeout	60s proxy abort to `EXTERNAL_BACKEND`

Optional `Authorization: Bearer` still increments `USAGE` KV when bound (`usage:<key>`, 30-day TTL).

Conceptual model


POST /api/run
validate body
fail
400
USAGE KV optional
PYODIDE_IN_WORKER
enabled
tryRunInWorker
disabled
ok
fail boot
200
proxyToExternal
no EXTERNAL_BACKEND
fetch

	503	EXTERNAL_BACKEND/run

Interface surface

Request body

```
{
  script: string;           // required non-empty
  data: Bar[];              // required non-empty OHLCV array
  mode?: 'interpret' | 'compile';
}
```

Invalid bodies → `400` `{ status: 'error', code: 'BAD_REQUEST', message }`.

Success / upstream

Proxy returns upstream status and body verbatim (JSON content-type, CORS origin header). Flask Pro API is the usual upstream.

Error codes (Worker-originated)

Code	HTTP	Meaning
`BAD_REQUEST`	400	Validation / size caps
`NO_KEY` / `INVALID_KEY`	401	Auth required and missing/unknown key
`RATE_LIMIT`	429	>30 runs/min for IP or key
`NO_BACKEND`	503	No external backend and Pyodide path off/failed path to proxy
(Pyodide)	200 body error	`tryRunInWorker` may return `{ status: 'error', error }` JSON

Usage metering

If `Authorization: Bearer …` and `USAGE` KV bound:

```
usage:{key} → count++, TTL 30 days
```

Internals

Path	Role
`worker/src/runtime.ts`	auth gate, rate limit, validate, meter, dispatch
`worker/src/http.ts`	shared CORS / JSON / error-envelope helpers (single source; values pinned by `worker/tests/http.test.ts`)
`worker/src/proxy-router.ts`	allowlisted route-table router for `/api/onchain/*` + `/api/market/*` (ordered; family fallbacks last — the allowlist is the security boundary)
`worker/src/rate-limit.ts`	shared per-IP/per-key minute-window limiter (`allowRate`)
`worker/src/pyodide_runtime.ts`	gated boot + `run_script` stub path
`worker/RUNTIME.md`	architecture plan for wheels / R2 / cold start
`worker/wrangler.toml [vars]`	`EXTERNAL_BACKEND`, `PYODIDE_IN_WORKER`, `REQUIRE_RUN_AUTH`

pyodide_runtime scaffold

When enabled:

- Lazy-loads Pyodide from jsDelivr `v0.29.5` (dev fallback).
- Production intent: R2 wheels + module-level cache (`RUNTIME.md`).
- Calls `run_script(script, bars)` via `runPythonAsync` — requires runtime bootstrap that is **not fully wired** like the browser engine's `pynescript_runtime.py` + micropip install.

Do not enable in production without completing the wheel pipeline and load testing CPU/memory limits.

Constraints (from RUNTIME.md)

Resource	Budget
CPU (paid)	30s limit; typical 500 bars ≪ 1s target
Memory	128 MB; Pyodide ~30 MB + wheel ~5 MB
Cold start	~3s first Pyodide boot (plan)

Roll-out flag

`RUNTIME.md` also mentions `RUNTIME_MODE=in-worker` as product language; **code checks `PYODIDE_IN_WORKER === 'enabled'`**. Prefer the code flag when configuring.

Invariants & edge cases

1. **Body stream** — request body is consumed once; proxy re-serializes the already-parsed object.
2. **CORS** — run responses set `Access-Control-Allow-Origin` from `pickOrigin` (see CORS).
3. **PWA path mismatch** — browser `server` engine posts to `{endpoint}/run`; Worker route is `/api/run`. Align via gateway rewrite or endpoint base path convention.
4. **No response caching yet** — README lists cache for identical `(script, data_hash)` as deferred.

Worked examples

Local proxy to Flask

```
# wrangler.toml [vars]
EXTERNAL_BACKEND = "http://127.0.0.1:5002"
PYODIDE_IN_WORKER = "disabled"
```

```
curl -sS -X POST http://127.0.0.1:8787/api/run \
  -H 'content-type: application/json' \
  -d '{"script":"//@version=6\nindicator(\"t\")\nplot(close)","data":[{"time":1,"open":1,"high":1,'
```

Force 503 for missing backend

Unset `EXTERNAL_BACKEND`, keep Pyodide disabled → expect `NO_BACKEND` message referencing `RUNTIME.md`.

Failure modes

Symptom	Fix
503 `NO_BACKEND`	Set `EXTERNAL_BACKEND` or finish Pyodide path
Proxy 502/connection refused	Flask not listening; wrong URL from Worker (use tunnel/public URL in CF, not localhost)
Pyodide enabled but errors	Wheel missing; CDN blocked; incomplete `run_script` bootstrap

See also

- Bindings
- Engines
- Auth (Bearer when gated; metering)
- CORS and origins
- AXIS CLI — `axis deploy worker` · `axis health`

DURABLE OBJECTS

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Durable Objects" description: "SessionDO WebSocket relay: one Binance upstream per session, fan-out to browser clients."

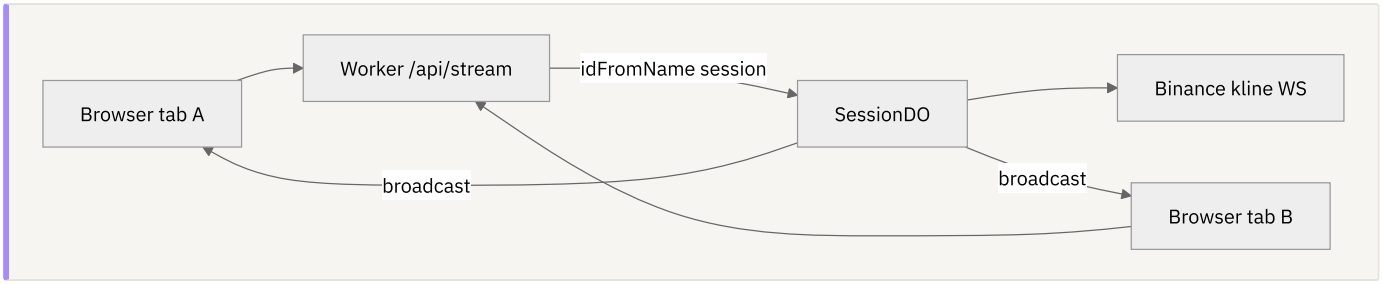
DURABLE OBJECTS

Abstract

`SessionDO` (`worker/src/durable-objects/session.ts`) is a **per-session WebSocket relay**. `McpBridgeDO` (`worker/src/mcp/bridge.ts`) is the **MCP control plane** for a connected PWA (see MCP server). Browsers connect to the Worker at `/api/stream`; the Worker pins a Durable Object by session name and the DO opens **one** upstream Binance kline socket, broadcasting payloads to connected clients.

Goal: survive browser per-origin WebSocket connection limits and share upstream subscriptions.

Conceptual model



Interface surface

Client → Worker

```
wss://<worker>/api/stream?session=<name>&symbol=BTCUSDT&interval=1m
```

`worker/src/index.ts`:

- Requires `env.SESSIONS` binding; else `503` `NO_DO`.
- `idFromName(session || 'default')`.
- Rewrites request to DO path `/ws?…` and returns upgrade response.

DO HTTP surface

Path	Role
`/ws`	Only path; requires `Upgrade: websocket` (else 426)

Query: `symbol` (default `BTCUSDT`), `interval` (default `1m`).

Client → DO messages (JSON)

Message	Effect
`{ "action": "subscribe", "symbol", "interval" }`	Rebind upstream if changed
`{ "action": "ping" }`	`{ action: "pong", t }`

DO → client

- Raw Binance kline JSON (same as direct venue WS).
- Status: `{ type: 'status', state: 'open'|'closed', url? }`
- Errors: `{ type: 'error', message }`

Example plugin

`public/plugins/example-cf-do-stream.js` maps kline fields to `Bar` and builds the Worker URL from config `endpoint`.

Internals

Topic	Detail
Upstream URL	`wss://stream.binance.com:9443/ws/{symbol}@kline_{interval}`
Client lifecycle	`acceptWebSocket`; on last client close → `closeUpstream`
Hibernation	Comment intent: hibernate when empty; implementation uses explicit close
Fan-out	`broadcast` to all `clients`

wrangler binding (often commented until ready)

```
# [[durable_objects.bindings]]
# name = "SESSIONS"
# class_name = "SessionDO"

# [[migrations]]
# tag = "v1"
# new_sqlite_classes = ["SessionDO"]
```

`SessionDO` is re-exported from `src/index.ts` for the Workers runtime class registry.

Invariants & edge cases

1. **Session key** — example plugin uses `symbol@interval` as session name so like-minded tabs share one DO.
2. **No auth on stream today** — treat as public relay of public market data; do not put secrets in query strings.
3. **Upstream hard-coded Binance** — not a generic multiprovider DO yet.
4. **Binding optional in local** — without `SESSIONS`, stream API fails closed with `NO_DO`.

Failure modes

Symptom	Cause
503 `NO_DO`	Binding commented / not deployed
426 expected websocket	Non-upgrade GET
No bars	Upstream block or wrong interval string
Stale symbol	Client did not send `subscribe` after change

See also

- Streams
- Bindings
- Plugin examples

DATA PLANE

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Worker data plane" description: "Script library API: /api/scripts, D1 schema, drafts, revisions, and in-memory fallback."

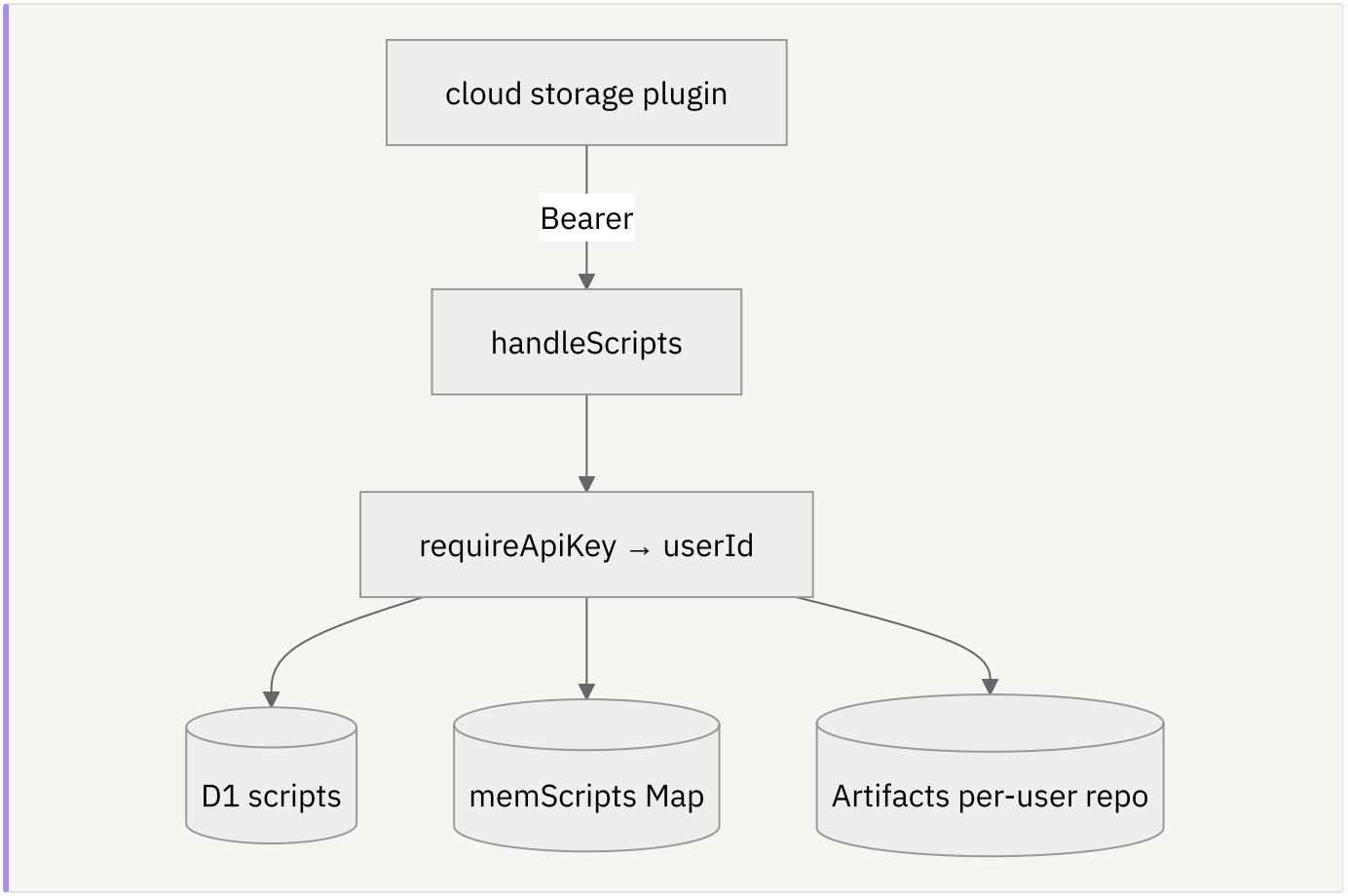
WORKER DATA PLANE

Abstract

The Worker **data plane** for user scripts is `worker/src/scripts.ts` — REST under `/api/scripts` with Bearer auth (auth). Persistence prefers **D1** (`env.DB`); without D1, per-isolate **in-memory** maps support local `wrangler dev`. With the optional `SCRIPT_ARTIFACTS` binding, every save additionally mirrors as a git commit to a per-key Artifacts repo — full history and `git clone` with no GitHub credentials (see below).

The PWA **cloud** storage plugin is the primary client.

Conceptual model



Git-native history (Cloudflare® Artifacts, no GitHub creds)

When `SCRIPT_ARTIFACTS` is bound (`axis setup artifacts`), every save/delete mirrors as a git commit to a per-key repo (`axis-scripts-<id12>`): `scripts/<id>.pine` plus a meta sidecar. History endpoints serve from `repo.log()` with real SHAs and fall back to `script_versions` when git is empty or unbound; responses carry `backend: 'artifacts-git'` when served from git. Mirrors run via `ctx.waitUntil` and never fail the save; identical re-saves dedupe against HEAD (no clone+push, no history spam).

Clone without any GitHub credential — the caller keeps using their AXIS API key while the Worker mints short-lived repo tokens:

```
REMOTE=$(curl -s $WORKER/api/scripts/_git -H "Authorization: Bearer $KEY" | jq -r .git.remote)
TOKEN=$(curl -s -X POST $WORKER/api/scripts/_git/token -H "Authorization: Bearer $KEY" \
  -H 'Content-Type: application/json' -d '{"scope":"read"}' | jq -r .git.token)
git -c http.extraHeader="Authorization: Bearer $TOKEN" clone "$REMOTE" axis-scripts
```

Setup:

```
axis setup artifacts --namespace axis-scripts   # writes [[artifacts]], verifies access
axis deploy worker
```

Artifacts is in closed beta: without access, `axis setup artifacts` fails the namespace check (re-run with `--skip-check` to keep the toml edit), and unbound Workers return `503 GIT_BACKEND_UNAVAILABLE` on `/_git` routes while CRUD keeps working on D1/memory.

Interface surface

All routes require valid API key unless noted.

Path	Methods	Behavior
`/api/scripts`	GET	List meta for `userId`
`/api/scripts`	POST	Create script
`/api/scripts/:id`	GET	Read full document
`/api/scripts/:id`	PUT	Upsert; honors `If-Match` revision
`/api/scripts/:id`	DELETE	Remove
`/api/scripts/_draft`	GET/PUT	Per-user draft buffer
`/api/scripts/:id/versions`	GET	Version history, newest first (git log when Artifacts bound, else `script_versions`)
`/api/scripts/:id/versions/:rev`	GET	Full document at a revision (git SHA or `rev_…`)
`/api/scripts/:id/restore`	POST	Resurrect `{revision}` as the new tip (recreates deleted scripts)
`/api/scripts/_git`	GET	Artifacts clone remote for the caller (503 when unbound)
`/api/scripts/_git/token`	POST	Short-lived clone token (`read` / `write`, TTL 60s–24h)

Document fields

Field	Notes
`id`	Client or server assigned
`name`, `description`, `path`	Meta
`content`	Pine source
`revision`	Opaque string (`rev_…`) for concurrency
`created_at` / `updated_at`	ms epochs (API may camelCase in JSON)

Concurrency

PUT with `If-Match: <revision>` rejects mismatched revisions (409 path covered in security tests). Fresh writes mint `newRevision()`.

Internals

D1 schema (`schemas/scripts.sql`)

```
CREATE TABLE scripts (
  user_id TEXT NOT NULL,
  id TEXT NOT NULL,
  name TEXT NOT NULL,
  description TEXT,
  path TEXT,
  content TEXT NOT NULL,
  revision TEXT NOT NULL,
  created_at INTEGER NOT NULL,
  updated_at INTEGER NOT NULL,
  PRIMARY KEY (user_id, id)
);

CREATE TABLE script_drafts (
  user_id TEXT PRIMARY KEY,
  content TEXT NOT NULL,
  name TEXT,
  updated_at INTEGER NOT NULL
);
```

Apply:

```
wrangler d1 execute pynescript --remote --file=schemas/scripts.sql
wrangler d1 execute pynescript --local  --file=schemas/scripts.sql
```

Binding name in `wrangler.toml` **must** be `DB` (code uses `env.DB`). Database name can be `pynescript`.

Partitioning

`userId = SHA-256(api_key).hex.slice(0, 32)` — raw keys are not stored as D1 partition ids.

Health feature flags

```
{
  "status": "healthy",
  "service": "worker-axis",
  "features": { "scripts": true, "d1": true/false, "keys": true/false }
}
```

Optional R2

`BUNDLES` R2 is reserved for indicator / pynescript wheels (`RUNTIME.md`). Not required for scripts CRUD.

Invariants & edge cases

1. **Memory store is not multi-isolate durable** — production needs D1.
2. **Drafts ≠ commits** — drafts are crash buffers; library list is separate.
3. **CORS** — scripts handler sets allow headers including `If-Match`.
4. **Cloud plugin dual-write** — browser still saves drafts to local storage.

Failure modes

Code / symptom	Meaning
401	Missing/invalid Bearer
404	Unknown script id for user
409	Revision conflict
503 `GIT_BACKEND_UNAVAILABLE`	`/_git` routes without the Artifacts binding (`axis setup artifacts`)
Empty list after deploy	Schema not applied / wrong DB id

See also

- Storage plugins
- Auth
- Bindings

AUTH

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Worker auth" description: "API keys (pn_...), fail-closed D1 without KV, admin token, Bearer, ALLOW_OPEN_KEYS, and /api/run gating."

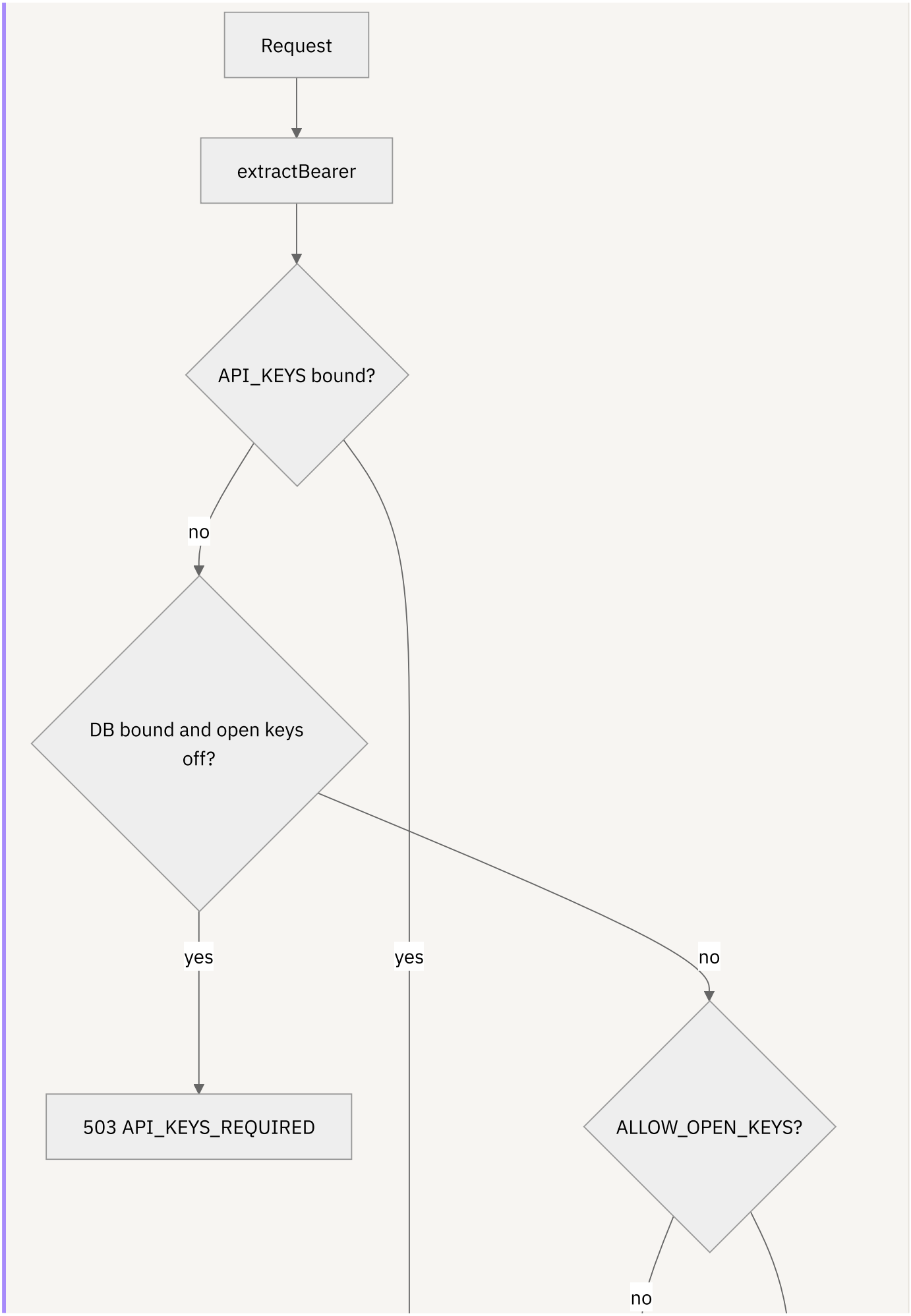
WORKER AUTH

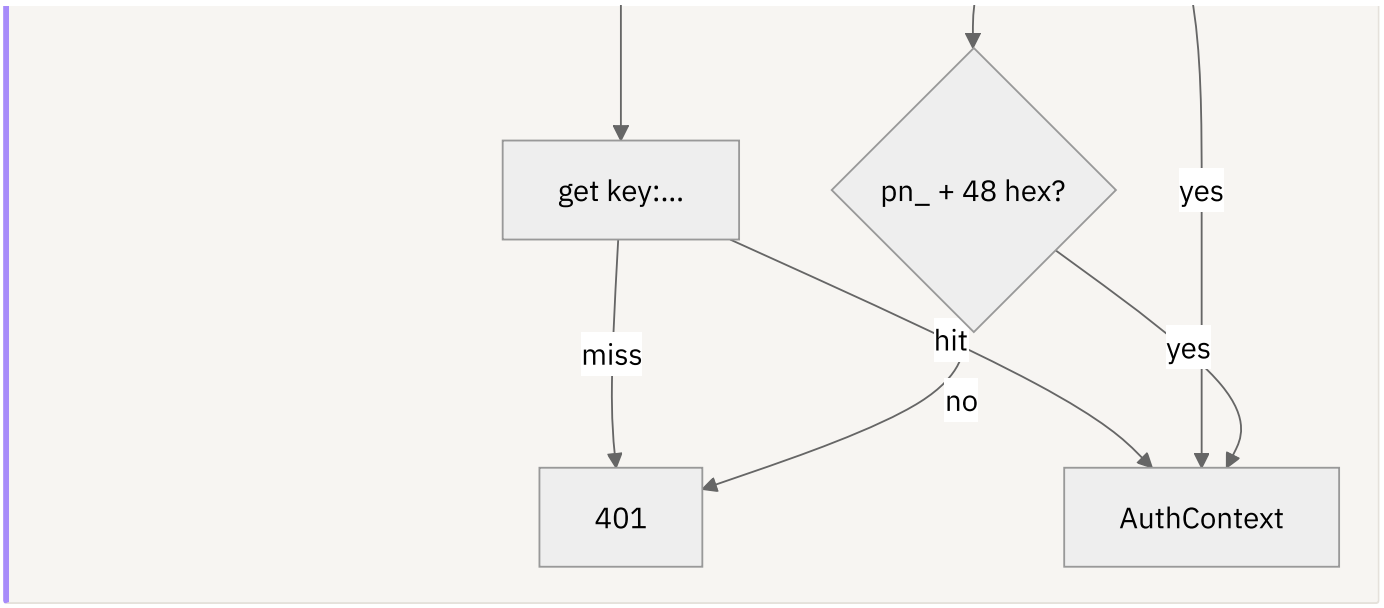
Abstract

AXIS Worker auth is **API-key based**, not session cookies. Keys are created by admins (`X-Admin-Token`), validated via KV when bound, and used as `Authorization: Bearer` on the script library and (when gated) on `/api/run`.

Core helpers: `worker/src/auth.ts`. Key CRUD: `worker/src/keys.ts`.

Conceptual model


Request
extractBearer
API_KEYS bound?
no
yes
DB bound and open keys off?
yes
yes
no
503 API_KEYS_REQUIRED
ALLOW_OPEN_KEYS?
no



Interface surface

AuthContext

```
{ key: string; userId: string; tier: string }
```

`userId` is a truncated SHA-256 of the key (stable partition id).

extractBearer

1. `Authorization: Bearer <token>`
2. Else query `?key=`

requireApiKey

Returns either `{ ok: true, ctx }` or `{ ok: false, status, code, message }`.

Code	HTTP	When
`NO_KEY`	401	Empty Bearer / `?key=`
`INVALID_KEY`	401	Unknown in KV / malformed shape
`API_KEYS_REQUIRED`	503	**Fail-closed:** D1 (`DB`) bound without `API_KEYS` KV and `ALLOW_OPEN_KEYS` off

Fail-closed with durable storage (2.0.1+)

When D1 is active but `API_KEYS` KV is **not** bound, inventable shape-only keys would partition real script data by attacker-chosen tokens with no mint/revoke path. In that configuration:

1. Only explicit `ALLOW_OPEN_KEYS=1` accepts any non-empty Bearer (local demos).
2. Otherwise respond **503** `API_KEYS_REQUIRED` — bind `API_KEYS` and mint via `/api/keys`.

MCP (`POST /mcp`)

Same `requireApiKey` as `/api/scripts`. Discovery `GET /mcp` is public. App-plane tools additionally need a PWA WebSocket on `/api/mcp/bridge`: mint a 30s one-time ticket with `GET /api/mcp/bridge?issue=ticket` (`Authorization: Bearer`), then connect `wss://…/api/mcp/bridge?ticket=…`. WS upgrade rejects the long-lived

`?key=` (`NO_KEY`). `?key=` remains HTTP `extractBearer` only (curl on `/mcp` / `/api/scripts`). See MCP server.

`/api/run` auth gate

From `worker/src/runtime.ts`:

Condition	Auth on `POST /api/run`
`API_KEYS` bound	**Required**
`REQUIRE_RUN_AUTH=1` (or `true` / `yes`)	**Required**
Neither	Optional (Bearer still meters when present)

Always rate-limited (30/min) regardless of auth — see Runtime.

Admin keys API (`/api/keys`)

Action	Auth	Behavior
create (`POST` or `?action=create`)	`X-Admin-Token` === `env.ADMIN_TOKEN`	Mint `pn_` + 48 hex, store `key:{key}` in KV (1y TTL) with tier
validate (`GET` or `?action=validate`)	Bearer or `?key=`	Tier + created_at if known

Tiers: `free` | `hobby` | `pro` | `team` | `enterprise`.

Without `ADMIN_TOKEN` set, **create always fails** (`isAdmin` false).

Do **not** leave `ADMIN_TOKEN = ""` in `[vars]`. That empty assignment is still a Worker binding, and Cloudflare® error **10053** rejects `wrangler secret put ADMIN_TOKEN` (`Binding name already in use`). `axis secret put ADMIN_TOKEN` comments the stub, deploys to drop it, then sets the secret.

CLI:

```
axis setup kv                 # create + bind API_KEYS
axis secret put ADMIN_TOKEN   # comments empty [vars] stub if present
axis keys create --tier hobby # prompts, or --admin-token / AXIS_ADMIN_TOKEN
axis keys validate --key pn_…
```

Without KV on validate: accepts well-formed `pn_[a-f0-9]{48}` as hobby (dev).

ALLOW_OPEN_KEYS

`wrangler.toml` defaults local demos to `"1"`:

> accept any non-empty Bearer key for `/api/scripts`

Never leave open in production. Prefer bound `API_KEYS` + real admin-minted keys.

Internals

Path	Role
`worker/src/auth.ts`	Bearer, hash, requireApiKey (fail-closed)
`worker/src/keys.ts`	Admin create / validate
`worker/src/runtime.ts`	Run gate + rate limit + usage meter
`worker/src/scripts.ts`	requireApiKey on library CRUD
`worker/src/git-oauth.ts`	Env `GITHUB_OAUTH_CLIENT_ID` / `GITLAB_OAUTH_CLIENT_ID` wins over body `clientId`

Key format

```
'pn_' + 24 random bytes as hex  // 48 hex chars
```

Invariants & edge cases

1. **Admin token is a shared secret** — rotate via wrangler secrets in production, not committed vars.
2. **KV record shape** — JSON `{ key, tier, createdAt }` under `key:${apiKey}`.
3. **No OAuth / cookies** — PWA stores the key in plugin config (`storage:cloud`).
4. **Stream DO unauthenticated** — public market data only.

Worked examples

Create a key (admin)

```
curl -sS -X POST 'http://127.0.0.1:8787/api/keys?action=create' \
  -H "X-Admin-Token: $ADMIN_TOKEN" \
  -H 'content-type: application/json' \
  -d '{"tier":"hobby"}'
```

Validate

```
curl -sS 'http://127.0.0.1:8787/api/keys?action=validate' \
  -H "Authorization: Bearer pn_…"
```

Failure modes

Symptom	Cause
403 create	Wrong/missing admin token
401 scripts	Open keys off + no KV + bad shape
503 `API_KEYS_REQUIRED`	D1 without `API_KEYS` KV and open keys off — bind KV + mint keys
401 on `/api/run`	`API_KEYS` or `REQUIRE_RUN_AUTH` set; missing Bearer
OAuth start uses attacker client	Env client id not set — set `GITHUB_OAUTH_CLIENT_ID` (env always wins over body)
Cross-user leakage tests fail	Partition hash regression

See also

- Data plane
- Bindings
- Runtime
- Security tests
- AXIS CLI — `axis secret put` · `axis setup oauth`

BINDINGS

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Worker bindings" description: "wrangler.toml vars, KV, D1, R2, Durable Objects — provision once, paste IDs. Worker script worker-axis."

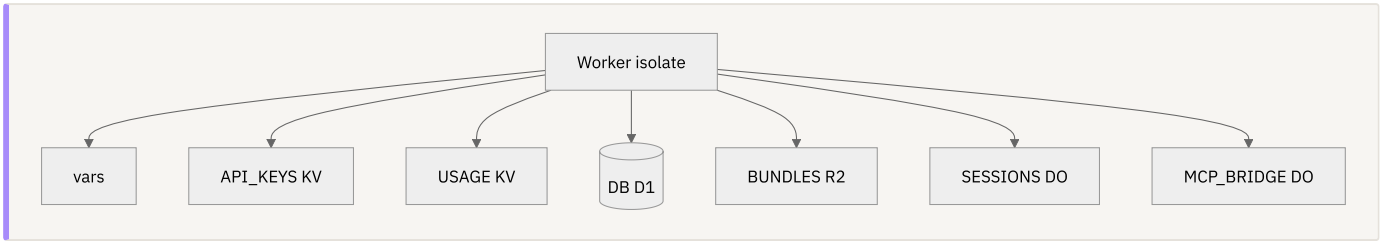
WORKER BINDINGS

Abstract

Bindings and vars for the AXIS Worker are declared in `worker/wrangler.toml` (copy from `wrangler.toml.example` via `axis setup worker`). Many production bindings ship **commented** so clone-and-dev does not require Cloudflare® resources; `wrangler dev` synthesizes local substitutes where possible.

Worker script name: `name = "worker-axis"`. Canonical host `worker.axis.hoox.sh`. Pages project is `axis`; canonical Pages host is `axis.hoox.sh`.

Conceptual model



Interface surface — Env

From `src/index.ts`:

```
interface Env {
  API_KEYS?: KVNamespace;
  USAGE?: KVNamespace;
  DB?: D1Database;
  BUNDLES?: R2Bucket;
  SESSIONS?: DurableObjectNamespace;
  MCP_BRIDGE?: DurableObjectNamespace;

  EXTERNAL_BACKEND?: string;
  ALLOWED_ORIGIN?: string;
  ADMIN_TOKEN?: string;
  PYODIDE_IN_WORKER?: string;
  ALLOW_OPEN_KEYS?: string;
  REQUIRE_RUN_AUTH?: string;
  GITHUB_OAUTH_CLIENT_ID?: string;
  GITLAB_OAUTH_CLIENT_ID?: string;
}
```

Vars (`[vars]`)

Var	Default (repo)	Role
`EXTERNAL_BACKEND`	`""`	Flask (or other) base URL for `/api/run` proxy
`ALLOWED_ORIGIN`	`https://pynescript.online`	Extra CORS allowlist (comma-separated); product hosts built-in
`ADMIN_TOKEN`	`""`	Shared secret for key minting
`PYODIDE_IN_WORKER`	`"disabled"`	Gate in-worker Python
`ALLOW_OPEN_KEYS`	`"1"`	Local demos; set `"0"` in prod
`REQUIRE_RUN_AUTH`	unset	`"1"` forces Bearer on `/api/run`
`GITHUB_OAUTH_CLIENT_ID`	unset	Preferred OAuth App id (wins over body)
`GITLAB_OAUTH_CLIENT_ID`	unset	Same for GitLab

Prefer **secrets** for `ADMIN_TOKEN` and production backends:

```
# CLI-first
axis secret put ADMIN_TOKEN
axis secret put EXTERNAL_BACKEND
# or: wrangler secret put …
```

Prod invariant: if `DB` (D1) is bound, also bind `API_KEYS` KV — otherwise script routes fail closed with `API_KEYS_REQUIRED`.

Provision recipes

KV

```
wrangler kv namespace create API_KEYS
wrangler kv namespace create USAGE
# paste ids into wrangler.toml [[kv_namespaces]]
```

D1

```
wrangler d1 create pynescript
# [[d1_databases]] binding = "DB"  (name MUST be DB)
wrangler d1 execute pynescript --remote --file=schemas/scripts.sql
```

Repo may already contain a `database_id` for the project's D1 — treat IDs as environment-specific when forking.

R2 (optional)

```
wrangler r2 bucket create indicator-bundles
# binding BUNDLES
```

Durable Objects

Uncomment bindings + migrations in `wrangler.toml`, then:

```
wrangler deploy
```

Class: `SessionDO` exported from `src/index.ts`.

Other wrangler settings

Setting	Value
`main`	`src/index.ts`
`compatibility_date`	`2024-11-01`
`compatibility_flags`	`nodejs_compat`
`[observability]`	`enabled = true`

Static PWA assets are **not** served by this Worker in the documented split: **Pages** serves `dist/` at `https://axis.hoox.sh`, Worker serves API/WS. Project name for Pages deploy is `axis`.

Deploy commands

```
cd worker
wrangler deploy

# Pages (repo root)
bun run build
wrangler pages deploy dist --project-name=axis
# or npm run deploy:pages from worker package
```

Makefile: `make worker-deploy`, `make pages-deploy` (note: some Makefile targets still point at legacy tree — prefer `dist/` from Vite build; see build and serve).

Invariants & edge cases

1. **Do not rename** CF project without migrating all binding IDs, custom domains, and CI.
2. **Local wrangler** auto-provisions mock KV/D1; production needs real IDs.
3. **`EXTERNAL_BACKEND=localhost` on CF** cannot reach your laptop — use a public tunnel or co-located VPS.
4. **Health** reports which optional bindings are present.

Failure modes

Deploy / runtime issue	Fix
Deploy fails placeholder KV	Uncomment only after create, or leave commented
Scripts empty	Apply SQL schema
Stream 503	Enable DO binding + migration
CORS fails prod origin	Set `ALLOWED_ORIGIN` to exact Pages origin

See also

- Cloudflare® DevOps
- Runtime
- CORS

MCP

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "MCP server" description: "Remote MCP for AXIS — Streamable HTTP /mcp, Bearer keys, Worker tools, live-app capabilities, McpBridgeDO."

MCP SERVER

Abstract

AXIS exposes a **Model Context Protocol** server on the Cloudflare® Worker so agents can **request and respond** across the data plane and a connected PWA.

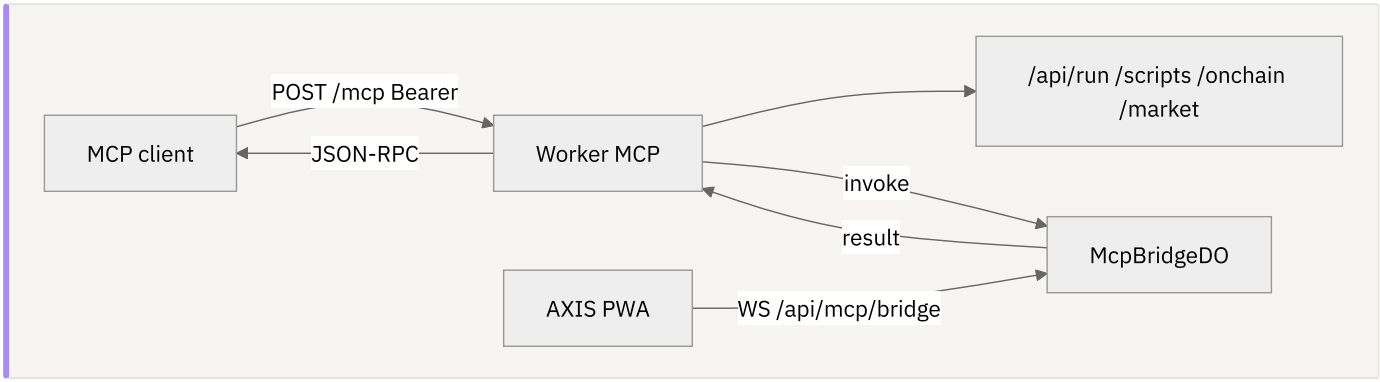
Plane	Live tab?	Tools
Worker	No	`axis_health`, `axis_request`, `axis_run`, `axis_scripts_*`, `axis_keys_*`, `axis_usage`, `axis_onchain`, `axis_market`
App	Yes	`app_invoke`, `app_get`, `app_set`, `app_capabilities`, `app_session_status`

Transport: **Streamable HTTP JSON-RPC 2.0** on `POST /mcp` (JSON body in / JSON body out). Protocol versions accepted: `2025-03-26`, `2025-06-18`, `2024-11-05`.

Auth: same **Bearer API key** as `/api/scripts` (`pn_…`). `GET /mcp` is public discovery. Production fail-closes when D1 is bound without `API_KEYS` KV. Rate limit: 60 RPC methods / minute / key (JSON-RPC batches charge per item, max 10; bodies above 1 MiB are rejected before parse).

End-user connect steps: MCP (agents).

Conceptual model



Endpoints

Canonical host: `https://worker.axis.hoox.sh`.

Path	Method	Auth	Role
`/mcp`	POST	Bearer	JSON-RPC: `initialize`, `tools/*`, `resources/*`, `prompts/*`, `ping`
`/mcp`	GET	public	Discovery JSON (server name, tool names)
`/mcp`	DELETE	—	No-op 204 (stateless)
`/.well-known/oauth-protected-resource`	GET	public	Bearer resource metadata
`/api/mcp/bridge`	WS	`?ticket=` (one-time, minted below) or `Authorization` header	PWA control plane → `McpBridgeDO`. Browser sockets cannot set Authorization; they never put the long-lived `pn_…` key in the query string.
`/api/mcp/bridge`	GET	Bearer	No `Upgrade` header → DO `/status`: `{ status, connected }` tab count (the app status bar polls this)
`/api/mcp/bridge?issue=ticket`	GET	Bearer	Mint a single-use WS ticket (`{ ticket, expiresIn }`, 30s TTL)

Health JSON includes `features.mcp: true` and `features.mcpBridge` when the DO is bound.

Bindings

`MCP_BRIDGE` Durable Object (`class_name = "McpBridgeDO"`, migration tag `v1` `new_sqlite_classes`). Declared in `worker/wrangler.toml.example`. App-plane tools return `NO_MCP_BRIDGE` until this is deployed. Cloudflare® no longer creates KV-backed DO namespaces (`new_classes`).

Session name is the API-key partition (`userId` = first 32 hex chars of SHA-256 of the key). Optional suffix `userId:label` is allowed; foreign session ids are ignored.

Worker tools

Tool	Role
`axis_health`	Same payload as `GET /health`
`axis_request`	Allowlisted Worker HTTP; returns `{ ok, status, body }`
`axis_run`	`POST /api/run` — `script` required; optional `data`, `ticker`, `timeframe`, `inputs`
`axis_scripts_list`	Cloud library metadata
`axis_scripts_get`	`{ id }` full script
`axis_scripts_put`	Create/upsert `{ name, content, id?, description?, path?, revision? }`
`axis_scripts_delete`	`{ id }`
`axis_keys_validate`	Current Bearer or `{ key }`
`axis_keys_create`	`{ adminToken, tier? }` — same as `X-Admin-Token`
`axis_usage`	Usage stub / KV counters
`axis_onchain`	`{ path, query? }` suffix after `/api/onchain`
`axis_market`	`{ path, query? }` suffix after `/api/market`
`app_session_status`	Connected PWA count for this key

`axis_request` arguments: `method` (`GET` default), `path` (required, starts with `/`), optional `query`, `body`, `headers` (`Authorization` is injected from the MCP session).

`axis_request` allowlist

Allowed: `GET /` `/health`; `POST /api/run`; `GET|POST /api/keys`; `GET /api/usage`; scripts CRUD (`/api/scripts`, `/_draft`, `/:id`, versions); `GET /api/onchain…`; `GET /api/market…`.

Blocked: `/mcp` recursion, `/api/stream`, `/api/git/oauth`, path traversal, unknown methods.

Resource `axis://allowlist` returns the live table.

App tools

Tool	Role
`app_capabilities`	Capability catalog from the connected tab
`app_invoke`	`{ capability, payload? }` — wait for JSON result (default 20s)
`app_get`	Sanitized snapshot; optional dotted `path`
`app_set`	Allowlisted dotted `path` + `value`

No tab → `SESSION_OFFLINE`. Timeouts → `TIMEOUT`. Snapshots redact keys matching `apiKey|secret|passphrase|token|password|authorization|cookie` and summarize OHLCV unless `includeBars: true`.

Capability ids (`app_invoke`)

Id	Payload	Mutates
`app.snapshot`	`{ includeBars? }`	no
`app.get`	`{ path?, includeBars? }`	no
`app.set`	`{ path, value }`	yes
`app.command`	`{ id }` palette command	yes
`app.shortcut`	`{ id }` shortcut id	yes
`app.event`	`{ name, detail? }` — `name` must start with `axis-`	yes
`chart.get`	—	no
`chart.set`	`{ symbol?, interval?, exchange?, source?, chartType? }`	yes
`chart.load`	`{ symbol?, interval?, source? }`	yes
`chart.reload`	—	yes
`chart.live`	`{ active? }`	yes
`chart.layout`	`{ op: get|grid|save|load|delete, mode?, name?, id? }`	yes
`chart.theme`	`{ presetId? }`	yes
`chart.type`	`{ type }`	yes
`chart.screenshot`	`{ scope?: price|panes|workspace, scale?: 1|2 }`	no
`chart.zoom`	`{ op: in|out|reset|pan, delta?, bars? }`	yes
`editor.get`	—	no
`editor.set`	`{ doc, open? }`	yes
`editor.run`	`{ doc? }`	yes
`editor.diagnostics`	—	no
`indicators.list`	—	no
`indicators.add`	`{ name, code, paneId? }`	yes
`indicators.remove`	`{ id }`	yes
`indicators.update`	`{ id, name?, visible?, code?, inputValues? }`	yes
`indicators.run`	re-run visible	yes
`alerts.list`	—	no
`alerts.create`	`{ name, kind, symbol?, params?, interval?, webhookUrl?, enabled? }`	yes
`alerts.update`	`{ id, patch }`	yes
`alerts.remove`	`{ id }`	yes
`watchlist.get`	—	no
`watchlist.add` / `remove`	`{ symbol }`	yes

Id	Payload	Mutates
`watchlist.lists`	`{ op: create|rename|delete|select, id?, name?, symbols? }`	yes
`library.list`	`{ prefix? }`	no
`library.read`	`{ id }`	no
`library.write`	`{ name, content, id?, description? }`	yes
`library.remove`	`{ id }`	yes
`results.get`	last run + strategy UI	no
`logs.get`	`{ limit?, source?, level? }`	no
`logs.append`	`{ message, level?, source? }`	yes
`logs.clear`	—	yes
`panels.get`	—	no
`panels.set`	`{ id, open?, dock?, chartOnly? }`	yes
`plugins.list`	—	no
`plugins.activate`	`{ kind: source|stream|engine|storage, id }`	yes
`theme.get` / `theme.set`	`{ theme?, presetId?, uiScale?, toggle? }`	set mutates
`workspace.export`	`{ includeBars? }`	no
`workspace.import`	`{ json }` or snapshot object	yes
`drawings.list`	—	no
`drawings.add`	`{ kind, points: [{time, price}…], style?, meta?, symbol?, id? }` — kind arity validated, symbol defaults to active	yes
`drawings.update`	`{ id, points?, style?, meta?, visible? }` — kind immutable	yes
`drawings.remove`	`{ id }`	yes
`drawings.clear`	`{ symbol? }`	yes
`drawings.tool`	`{ tool }`	yes
`settings.get`	`{ key? }` — sanitized settings, secrets never exposed	no
`settings.set`	allowlisted patch (`endpoint` , `engine` , `interval` , `historyBars` , `refreshSec` , `live.*` , `uiScale` , `autoload` , label/strategy/telemetry prefs); interval/history changes reload	yes
`status.get`	status bar + telemetry	no

`app_set` paths: `symbol` , `interval` , `source` , `engine` , `exchange` , `theme` , `chartType` , `uiScale` , `live.active` , `live.preferAfterLoad` , `editor.doc` , `editor.open` , `watchlist.open` .

Resources and prompts

URI	Contents
`axis://health`	Health JSON
`axis://capabilities`	Tool names + planes
`axis://allowlist`	`axis_request` paths
`axis://session`	Bridge status for this key

Prompts (seed the agent; they still call tools):

Name	Args
`run_script`	`script` (required), `symbol?`, `interval?`
`debug_last_run`	—
`analyze_strategy`	—
`load_symbol`	`symbol`, `interval?`
`audit_workspace`	—

Client config

```
axis mcp config --key pn_…
axis mcp --key pn_…                 # stdio proxy
```

```
{
  "mcpServers": {
    "axis": {
      "url": "https://worker.axis.hoox.sh/mcp",
      "headers": { "Authorization": "Bearer pn_…" }
    }
  }
}
```

Local wrangler: `http://127.0.0.1:8787/mcp` with `ALLOW_OPEN_KEYS=1` still requires a non-empty Bearer.

Errors

Code	Where	Meaning
`NO_KEY` / `INVALID_KEY`	HTTP 401	Missing or unknown Bearer
`API_KEYS_REQUIRED`	HTTP 503	D1 without KV
`RATE`	HTTP 429	60/min/key
`MCP_ALLOWLIST`	tool	Path not allowed
`NO_MCP_BRIDGE`	tool	DO unbound
`SESSION_OFFLINE`	app tool	No PWA socket
`TIMEOUT`	app tool	Tab did not reply in 20s
`UNKNOWN_CAPABILITY` / `PATH_DENIED`	app	Bad id or set path
`-32601`	JSON-RPC	Unknown method

Implementation

Path	Role
`worker/src/mcp/handler.ts`	Streamable HTTP JSON-RPC
`worker/src/mcp/catalog.ts`	Tools / resources / prompts
`worker/src/mcp/allowlist.ts`	`axis_request` gate
`worker/src/mcp/proxy.ts`	In-isolate dispatch to existing handlers
`worker/src/mcp/bridge.ts`	`McpBridgeDO`
`src/mcp/`	PWA host, snapshot, dispatch, WS client
`packages/cli/src/commands/mcp.ts`	`axis mcp`

In-page: `window.__AXIS_MCP__.invoke('chart.get')`. Tests: `worker/tests/mcp-handler.test.ts`, `tests/mcp-dispatch.test.ts`, `packages/cli/tests/mcp.test.ts`.

This is a **stateless** MCP handler (no deprecated `McpAgent` session DO). App state lives in the PWA + `McpBridgeDO`, not in an MCP session id.

See also

- MCP (agents)
- Auth
- Durable Objects
- Bindings
- AXIS CLI